

Astrophysical Recipes (Second Edition)

The art of AMUSE

Simon Portegies Zwart, Steve McMillan and Steven Rieder

Appendix C

Programming Primer

To be a programmer is to develop a carefully managed relationship with error.

—Ellen Ullman

In this book, we assume that you are familiar with the Linux operating system, with the Python programming language, and with the use of `git`. If this is not the case, we present here a very basic primer about them. Linux is a UNIX-based operating system designed in 1991 by Linus Torvalds. Python is a third-generation computer language designed in 1991 by Guido van Rossum. Git is a source code management system that began in 2005 as a development system for Linus Torvalds’s Linux kernel.

C.1 Linux

The Linux operating system was originally designed for microcomputers.¹ It is composed of a number of functional components that together form the operating system. These components include (see <http://linux.com>):

The **bootloader**, the software that manages the startup process of your computer.

The **kernel**, the core of the system that manages the CPU, RAM memory, and peripheral devices.

Daemons, background services that manage printing, sound, scheduling, etc., that either startup during boot, or when you log in to the desktop.

The **shell**, which manages commands to control the computer. Commands are typically typed into a terminal with a text interface, or contained in scripts.

¹“Microcomputer” refers to the type of machine we nowadays call a workstation. Minicomputers used to be the midlevel machines that we now call cluster computers or Beowulfs (Sterling 2002), not to be confused with the Raspberry Pi processor board, which is sometimes also referred to as a minicomputer.

The **graphical server** (or `X server`), the subsystem that displays graphics on your monitor.

The **desktop environment**, the customizable user interface. There are many choices, such as Gnome, KDE, Cinnamon, Enlightenment, etc.

Applications (apps), the programs that run on your computer. In this book, you write your own applications using AMUSE, but you can download applications or write them in other languages too. One application you will be using a lot is a text editor. This can be emacs (Ciccarelli 1978), Pico, vim, or any of a number of others. Choice of editor is squarely in the category of religion and politics, and we make no recommendation. Another application you will be using a lot is the Python interpreter (van Rossum 1995a, b, c, d).

You generally communicate with your computer via the desktop environment. Most modern operating systems have desktop environments, but the sophisticated shell and the programming style it enables make Linux (and its cousin UNIX) rather unique. Some other operating systems have command-line interfaces, but they are often undeveloped and almost completely useless. Note that there is not a single Linux, but many flavors, including Fedora, Ubuntu, RedHat, SuSE, Debian, Gentoo, and many more (see <http://distrowatch.com/>). The practical differences are generally not so large, although true OS geeks swear by their preferred flavor. MacOS is not Linux, but is based on FreeBSD, a version of UNIX, and presents a programming environment essentially equivalent to that found in Linux.

The shell allows you to send commands directly to your computer, or launch scripts to automate this process. Some typical Linux commands, in no particular order, include:

```
%> man -t ascii | ps2pdf - > ascii.pdf makes a PDF file of a
manual page.
%> apropos whatis combines two commands: apropos, which lists
alternatives on a specific command, and whatis, a rudimentary help
function.
%> which python shows the full path name of the command Python.
%> mkdir amuse creates a directory with the name amuse.
%> ls amuse lists the contents of directory amuse.
%> cd amuse go to directory named amuse.
%> find . -name *.py finds all files with the extension .py in all
subdirectories.
%> locate sun_venus_earth.py finds the file sun_venus_earth.py.
%> tar -x dir | gzip make a zip-compressed archive of the directory dir.
%> sort -u file1 file2 make a union of unsorted files.
%> gzip < /dev/cdrom > cdrom.iso.gz saves a copy of data from a
CD-ROM.
%> ps -aux shows a list of all processes running on your computer.
%> top shows an interactive list of your current processes.
%> grep MemTotal /proc/meminfo shows the total random access
memory in the system. (Note that macOS does not have a /proc filesystem.)
```

```
%> cat /proc/cpuinfo displays the contents of the file named /proc/
cpuinfo.
%> tail -lines=10 filename shows the last 10 lines of the file named
filename.
%> less filename displays the content of the file filename, page by page.
```

Note the use of the operators “<” (read from a file or the keyboard), “>” (write to a file or the computer screen), and “|” (connect the output of one command to the input of another).

C.1.1 A Little *awking* and *grepping*

The Linux shell includes a large number of handy utilities, which can help you work more efficiently with the operating system. The commands *awk* (see <http://www.gnu.org>) and *grep* (Kernighan 1984) are particularly useful for rudimentary data processing.

- *awk* is a scripting language designed to manipulate ASCII text documents and automate the extraction of data. It was developed in the 1970s by Alfred Aho, Peter Weinberger, and Brian Kernighan (Aho et al. 1987).
- *grep* is a utility run from the Linux command line that searches for regular expressions in plain-text data sets.

Typically, *grep* is used to search and catch certain phrases, whereas *awk* is used to manipulate them. Knowing some basic *awk* and *grep* commands will save you valuable time, and we strongly recommend that you learn how to work with them. You might begin with

```
%> man awk
```

and

```
%> man grep
```

both of which will provide more information than you need right now. These utilities make for a powerful experience in the command shell.

Another handy command-line tool is the stream editor *sed*, which is very efficient at manipulating a data stream (from a file or another program) on the fly.

C.2 Git

While working through this book, you are likely to write source code for several numerical experiments. *Git* is an excellent source code management system that lets you keep track of and manage the different versions of your code. Tutorials and complete documentation are readily available online at many locations, but we recommend <https://www.atlassian.com/git/tutorials/>.

Probably the simplest way to start using *git* is to create an account on one of the online *git* brokers, such as bitbucket.org, github.com, or sourceforge.net.

net. These sites are widely used and free. There are many more sites, some providing better services and enhanced security, but at a cost. The details vary, but after you create an account, you will generally be guided through the steps needed to create a new minimal project, which you can then clone to your own computer and populate with your own files, as described below.

Alternatively, you can make and manage your own `git` repository. First create a directory on the computer where you want the repository to reside. The following commands create a directory containing an empty `git` repository.

```
%> mkdir project-home
%> cd project-home
%> git init -bare
%> git update-server-info
```

Now you can update the repository with your project source code, copied from another computer or another directory on the same computer. We assume you have placed it in the local subdirectory `project`. You can add the code to the repository as follows.

```
%> cd project
%> git init
%> git add *
%> git commit -m "First version of my code"
%> git remote add origin spz@eucalypta.strw.leidenuniv.
nl:project-home

%> git push -u origin master
```

Here, `spz@eucalypta.strw.leidenuniv.nl:project-home` refers to your `git` server's URL and the particular directory on that server where the project resides. Of course, you do not want to store your data on the first author's computer at the Leiden Observatory in the Netherlands, so you had better create your own!

Now, suppose you want to access the code on another computer, maybe your laptop, make changes, and upload them to the master repository. To do this, you first have to make a local directory for your code and clone the master copy of the code:

```
%> git clone spz@eucalypta.strw.leidenuniv.nl:project-
home project
```

Now, you can make changes to the source code using your favorite editor and upload them:

```
%> cd project
%> ...(edit files)...
```

```
%> git add filename.cpp
%> git commit -m "Small change made in filename.cpp"
%> git push origin master
```

Your changes are now added to the master repository, but that does not mean that they have been propagated to other clones of the code. Suppose your friend's computer also has a working copy of the repository. She is still working with the version from before the changes you made, but can update her local version with the commands

```
%> cd project
%> git pull
```

Now she also has the latest version.

This all sounds very nice, although maybe a bit arcane, but believe us, it is a great improvement over exchanging source code by e-mail or using other version control systems. More information on `git` can be found at <http://rogerdudler.github.io/git-guide/>.

C.3 Python

If you learned to program computers in the dark ages, like two of the authors, Python (van Rossum 1995a, b, c, d) is a breath of fresh air. Almost like air travel in the 1980s: few restrictions, and nothing to declare! Python is easy to learn and it is almost impossible to write obfuscated code in (in contrast to C: see <http://www.ioccc.org/>). In this section, we review a few basics you should know about Python in order to use this book. This section will be as short as possible because there are a great many excellent books written about the language. We recommend especially the many books about the language by Hans Petter Langtangen, and specifically *Python Scripting for Computational Science* (Langtangen 2009).

Python is a procedural language, which means that it works with lists of instructions that tell the computer what to do with input and data. Other examples of procedural languages include C (Kernighan & Ritchie 1988) and FORTRAN (American National Standards Institute 1966). Some procedural languages incorporate abstract data types that allow objects can be created and manipulated. Examples of these object-oriented languages include C++ (Stroustrup 2013) and Java (Gosling et al. 1996, 2014). Python is also an object-oriented language, and includes some other interesting features drawn from C++ and Lisp.

Python is a typed language—variables have specific types that can only be combined in certain ways—but unlike the “scientific” programming languages listed above, types are not declared at compile time, but instead are inferred from context as needed. This makes Python both very flexible and much slower than C++ or FORTRAN. Indeed, the whole point of AMUSE is to combine the flexibility of Python with the speed of compiled procedural languages. Python also supports reflection—the program structure can be modified during execution.

You generally start a Python session on the command line by invoking the interpreter

```
%> python
```

or, for the somewhat more practical interactive shell interpreter,

```
%> ipython
```

Both are basically the same language (Perez & Granger 2007). If you wish, you can try the Jupyter notebook environment (Kluyver et al. 2016), which can be found at <http://jupyter.org/>. This is a browser-based support structure on top of the usual Python interpreter, and has a number of powerful and interesting features.

C.3.1 Python Types and Printing

As a procedural language, a Python script is composed of a list of instructions. One of the simplest is a scalar assignment:

```
>>> star = 5
```

(here and below `>>>` represents the interactive Python prompt), which here assigns the integer 5 to the variable `star`. If we had wanted to use a floating point number, we would have said

```
>>> star = 5.0
```

The instructions can contain arithmetic operations

```
>>> other_star = 4*star - 2
>>> print(other_star)
18
```

Note that the Python interpreter decides a variable's type at run time, and the type can change from line to line, e.g.,

```
>>> star = 5
>>> a = star
>>> star = 5.0
>>> print(a, star)
5. 5.0
```

One “feature” of Python 2 was that dividing an integer by an integer returns an integer. For example, if you divided two integers and printed the result, you would get

```
>>> print 1/2
0
```

This is very familiar to C and FORTRAN programmers, and is even very useful once you understand that it happens, but it is a never-ending source of confusion for beginners. Python 3 has eliminated this feature (to the chagrin of the old-timers), and the same code would produce the following:

```
>>> print(1/2)
0.5
```

while the former behavior can be reproduced with the `//` operator:

```
>>> print(1//2)
0
```

In our opinion, it is generally better (meaning that it produces more predictable code) to write the decimal point explicitly:

```
>>> print(1./2.)
0.5
```

To be sure about the type of variable, you can always query its type explicitly:

```
>>> type(star)
<type 'int'>
```

We have used the function `print` several times above. In Python 2.x, we could have eliminated the parentheses:

```
>>> print 1./2.
0.5
```

but in Python 3 they are required. Integer division and the `print` statement are the two most noticeable differences between Python 2 and Python 3. Because AMUSE was originally designed and implemented using Python 2.7, we usually assumed that

integer division yields an integer and used print statements without parentheses in the AMUSE source for the first edition of this book. In this edition, we have switched fully to Python 3.

C.3.2 Lists

Variables of different types can be combined into lists, e.g.

```
>>> stellar_list = [star, other_star, 1/2, 1./2.]
```

which will give

```
>>> print(stellar_list)
[5, 18, 0, 0.5]
```

Lists are nice. They can contain anything you like. Individual elements can be accessed by using their position in the list. Like C, but unlike FORTRAN, Python counts from zero, so the list here runs from element 0 to element 3.

The length of a list can be queried by the function `len()` with the list as an argument:

```
>>> len(stellar_list)
4
```

The last element of a list can also be accessed using by the index `-1`:

```
>>> print(stellar_list[-1])
0.5
```

You can check this by comparing the last element of a list referenced in two different ways:

```
>>> stellar_list[-1] == stellar_list[len(stellar_list)-1]
True
```

The value `True` indicates that the two expressions are indeed identical.

A list is really a class (see below). This means that a number of operations are defined to work with or on lists. For example, to add a new element to a list

```
>>> stellar_list.append(100)
```


or sort a list

```
>>> stellar_list.sort()
>>> print(stellar_list)
[0, 0.5, 4, 14, 100]
```

or remove an element from a list

```
>>> stellar_list.pop(0)
>>> print(stellar_list)
[0.5, 4, 14, 100]
```

Lists also have operators, for example the `+` operator that enables you to concatenate several lists

```
>>> stellar_list + [3, 2, 1]
[0.5, 4, 14, 100, 3, 2, 1]
```

You can do a lot more with lists, but by now you probably get the idea.

C.3.3 Dictionaries

Making a hash table in Python is easily accomplished by using a dictionary. This works as follows

```
>>> stellar_mass = {'Betlegeuse': 8, 'Rigel': 21, 'Spica': 10.25}
>>> for name, mass in stellar_mass.iteritems():
...:     print('star: %s has mass: %s' % (name, mass))
...:
star: Rigel has mass: 21
star: Spica has mass: 10.25
star: Betlegeuse has mass: 8
```

Each of the entries Betlegeuse, Spica, and Rigel are now associated with a value. Just as in a list, this value can be anything: a parameter, a function, or even a class.

C.3.4 Flow control

A simple Python program is usually composed of conditionals and loops, just as in any other procedural language. The main flow control elements in Python are `if`, `for`, and `while`.

A typical `if` statement chooses between two or more alternatives:

```
>>> if star == 6:
...:     print(star)
...:else:
...:     print(other_star)
...:
18
```

Other comparison operators (e.g. `<`, `<=`, `>`, `>=`, etc.) can be used in place of `==`. One slight peculiarity of Python is that it also includes the operators `is` and `is not`. You might think that `==` and `is` should mean more or less the same, but actually `==` compares the value of two expressions, whereas `is` tests if the two objects are the same:

```
>>> 1 == True
True
>>> 1 is True
False
```

The `for` loop has a running parameter (here called `i`) that increases from a value 0 up to some value:

```
>>> for i in range(3):
...:     print(i)
...:
0
1
2
```

In this case, we simply print the variable `i`. The loop stops automatically when the objective is reached. Note that, in consistent Python usage, `range(3)` actually returns the list `[0, 1, 2]`. Python recognizes which instructions belong to the loop by indentation. In most AMUSE scripts, we adopt four spaces of indentation, although sometimes, if the lines become long and the number of inner loops large, we use two. Other languages use special symbols to indicate the start and the end of a loop, such as braces in C or the words `DO` and `END DO` in FORTRAN. (C and FORTRAN programmers generally hate Python's strict reliance on indentation to indicate structure.)

A `while` loop continues until some Boolean expression is no longer true. The previous loop could have been written

```
>>> i = 0
>>> while (i < 3):
...:     print(i)
...:     i += 1
...:
0
1
2
```

To be honest (as is also the case in C and FORTRAN), we do not really need both `for` and `while` loop constructs—one can always be written in terms of the other, but tradition is a powerful thing. Unlike C or C++, Python has no `case` or `switch` statements. This is not a problem because again these functionalities can be constructed as needed using the available flow control operators.

C.3.5 Functions

Python allows the user to define functions. They can take and return any argument (including other functions). The purpose of a function is to prevent code duplication and to make your code as modular as possible. A simple function in Python might look like

```
def factorial(n):
    if n < 1:
        return 1
    else:
        return n*factorial(n-1)
```

(Note: you should never write the factorial function this way in a real script!)

A more useful function that calculates the radius of a star in solar radii, given its mass in solar masses, might be:

```
def stellar_radius(mass=1.0):
    radius = mass**0.8
    return radius
```

Note the use of a default value for the argument. You might want to save this handy function in a file called `calculate_stellar_radius.py` for later use. Then, from another script (or interactive Python session) you could load this function and use it:

```
from calculate_stellar_radius import stellar_radius
r = stellar_radius(2)
print(r)
```

We recommend trying to keep functions small, less than a page in length if possible, and focused on a single operation.

C.3.6 Importing packages

In an earlier example (in Section C.3.5), we imported a function from a file we had just created. Many functions and entire packages are already written, and you should always try to avoid rewriting anything that has already been written, except perhaps for pedagogical purposes. Just as we import the `amuse.lab` package, you can import other packages. Important packages include `numpy` (van der Walt et al. 2011), `scipy` (see for example Rojas et al. 2015), `math` for numerical, scientific, and mathematical extensions of the language. There are also packages for plotting, such as `pyplot`, machine learning `pyML`, MCMC (e.g., `emcee`; Foreman-Mackey et al. 2013), `Astropy` (The Astropy Collaboration et al. 2013), and many more.

C.3.7 Classes

A class is a data container with associated operators and functions to manipulate the data it contains. An example of a widely used class in AMUSE is the `Particle` set. Here, we illustrate the use of classes by creating a simple `Body` class that can form the basis for more complex data structures. Code example C.1 illustrates how to declare a `Body` base class.

This class is not complicated—it requires only one initialization parameter, the mass of the object, contains two class variables, called `type` and `mass`, and two class methods (functions), one to initialize the class itself and one to return its contents as a string. A more elaborate example of the use of classes in AMUSE can be found in Section 9.3.2.

A derived class is a class that inherits the properties of a base class and extends its functionality. Code example C.2 presents two examples, for a planet (called `Planet`) and for a star (called `Star`), both derived from `Body`. (Class declaration names in Python conventionally start with a capital letter.) They share the `Body` class variable `mass` and the `Body` member functions. Both, however, extend the class, `Planet` by adding the orbital parameters semimajor axis and eccentricity, and `Star` by adding a radius. In addition, they both override the `type` class variable, to “planet” and “star”, respectively, and modify the string returned by `__str__`. We can now manipulate these classes, make declarations, put them in arrays, give them to functions as arguments, etc.

A useful aspect of classes is the ability to make compound classes that themselves are classes. This is illustrated in Code example C.3. We could further develop this class structure by making it more object-oriented, but that would be beyond the scope of this book, particularly because there are many books that do an excellent job in explaining classes and inheritance.

Classes are nice and versatile. They make programming a lot more fun, and we strongly encourage you to explore their potential. Despite these recommendations,

```

2 class Body:
3     def __init__(self, mass):
4         self.type = "body"
5         self.mass = mass
6
7     def __str__(self):
8         return f"{self.type} mass = {self.mass}"

```

Code example C.1.: A base class.

```

14 MassEarth = 3.0e-6
15 RadiusEarth = 0.01
16
17
18 class Planet(Body):
19     def __init__(self, mass, semimajor_axis, eccentricity=0.0):
20         Body.__init__(self, mass)
21         self.type = "planet"
22         self.semi = semimajor_axis
23         self.ecc = eccentricity
24         self.radius = RadiusEarth * (mass / MassEarth) ** 0.55
25
26     def __str__(self):
27         return f"{self.type} mass = {self.mass}, a = {self.semi}"
28
29
30 class Star(Body):
31     def __init__(self, mass):
32         Body.__init__(self, mass)
33         self.type = "star"
34         self.calculate_radius()
35
36     def __str__(self):
37         return f"{self.type} mass = {self.mass}, radius = {self.radius}"
38
39     def calculate_radius(self):
40         self.radius = self.mass**1.2

```

Code example C.2.: Derived class fragments.

in this book we have largely refrained from using classes, mainly to avoid going too deeply into unfamiliar territory.

C.3.8 Operator Overloading

The redefinition in the previous section of member functions with double leading and trailing underscores is a form of overloading. The member functions

```
def __init__(self):
```

```

47 class PlanetarySystem(Body):
48     def __init__(self, star, planets):
49         self.type = "planetary system"
50         self.star = star
51         self.planets = planets
52
53     def print(self):
54         print(self.star)
55         for pi in self.planets:
56             print(pi)

```

Code example C.3: A compound class.

and

```
def __str__(self):
```

defined in Code example C.1 overload the previously existing member functions of the same names. Aside from these predefined functions, many other functions can be overloaded, including arithmetic and other operators. You can even define your own operators and give them specific functionality. In AMUSE we have overloaded the pipe command, |, to add a unit to a quantity. The `Astropy` package overloads the `*` operator for much the same purpose.

For example, in the earlier example of the `Body` base class, we could overload the `+` sign for adding a planet to a star. This could be realized by adding the following method to the `Star` class

```

def __add__(self, planet):
    self.mass += planet.mass
    self.calculate_radius()
    return self

```

In an executable script, we could then write

```

s = Star(mass=1)
p = Planet(mass=0.01)
t = s + p
print(t)

```

Which would result in

```
star mass = 1.01, radius= 1.01201196814
```

This simply returns a `Star` with a larger mass. Obviously, we would want to do more, perhaps adding functionality to compute and/or evolve the planet’s orbit, but the basic procedure should be clear. Quite a number of operators can be overloaded in this fashion.

C.3.9 Properties

You will find the line

```
@property
```

in a few places in the text, and in numerous class definitions in the AMUSE source code. This construction is unfamiliar to many casual Python users, so we present a brief indication here of what it means and how it works.

The `@property` line preceding a class method definition in Python is a concise way to build a data descriptor that triggers a function call when an attribute is accessed. What does that mean? For example, in the `Star` class, we might add the method

```
@property
def core_mass(self):
    return 0.3*self.mass
```

Without the `@property` line, we would access the core mass with

```
s = Star(1)
mc = s.core_mass()
```

With the `@property` line, the second line above would become

```
mc = s.core_mass
```

Why is it important to turn a function call into an attribute? In AMUSE, the principal reason for doing this is to make access to class data more homogeneous. For example, in the `Particles` class, calculating the `kinetic_energy` and `potential_energy` are done internally using functions, but it is convenient to use the `@properties` syntax to make these quantities accessible as class variables.

This may still seem like a minor syntactic change, but in Python this approach is part of a broader class of operations that control access to internal class data that should not be accessed directly. In other object-oriented languages, such as C++, setting and getting private data in a class is commonly realized using a `set` or a `get` function. This allows the designer of a class to control how the class data are modified and used. In Python, which has “best-practice” conventions for limiting

data access but does not actually have private data, this is best done via the `@property` paradigm and related constructs.

The syntax allows the result of a complex calculation to appear as a class attribute, as we have just seen, but it can also be used to prevent users from directly setting variables with “side effects.” For example, changing the mass of a star with a planetary system should also adjust the orbital elements of the planets orbiting it. Hence, we should not allow the mass to be modified directly, but instead intercept that attempt with a function that both changes the mass and modifies the orbits. However, since AMUSE does not use this extended functionality, we must stop the discussion here and refer the interested reader to the extensive Python documentation on the subject.

C.3.10 Testing and Debugging Python Scripts

Every script that is intended for anything other than testing should be tested, and the test scripts should be tested too. From personal experience, we know that any program we write is likely to have some serious shortcomings (aka bugs) if it is not thoroughly tested. Tests can be simple, but they should test fundamental functionalities of every script. It is not uncommon to spend more time writing tests than writing production code. In the end, writing extensive test functions can speed up the development of the production code enormously, and, once finished, the code is much more likely to work as intended. Time devoted to writing test scripts is always time well spent.

A test code starts by importing the relevant Python module

```
import unittest
```

Within the script to test, `test_get_posvel_from_binary_elements.py`, we add a test case, for example something simple like:

```
def test_get_posvel_from_binary_elements():
    a = 1 | units.au
    try:
        r, v = get_posvel_from_binary_elements(a, 0.0, 1)
        assert False
    except:
        pass
    r, v = get_posvel_from_binary_elements(a, 0.0, 1 | units.MSun)
    assert r[0] == a
```

If the `assert` statement fails, the test fails, and you should start debugging the relevant routine. You can test the script by calling:

```
%> pytest test_get_posvel_from_binary_elements.py
```


You should create tests for every routine you write. Give the test routine the same name as the routine that performs the particular test, and run `pytest` to perform the test.

Making and running test scripts will help you to write correct and (almost) bug-free code. However, the time will probably come when you have to start a debug session (see Appendix B.1.7). Debugging is a necessity at every level of programming. It is no fun to hunt for the source of a segmentation fault². A run-time debugging tool may provide much needed relief. In Python, this can be done by importing the `logging` library, after which it is possible to run a script in the debugger. This is done as follows:

```
import logging
logging.basicConfig(level=logging.DEBUG)
logging.getLogger("code").setLevel(logging.DEBUG)
```

There are few hard and fast practical rules on debugging. Pin down the bug using lots of `print` statements, wade through the data they produce to augment the debugger data, and good luck!

References

- Aho, A. V., Kernighan, B. W., & Weinberger, P. J. 1987, *The AWK Programming Language* (Boston, MA: Addison-Wesley)
- American National Standards Institute 1966, *ANSI Fortran X3.9-1966* (New York: American National Standards Institute)
- Ciccarelli, E. C. 1978, *An Introduction to the EMACS Editor*, AI Memo 447, *MIT*
- Foreman-Mackey, D., Hogg, D. W., Lang, D., & Goodman, J. 2013, *PASP*, **125**, 306
- Gosling, J., Joy, B., & Steele, G. L. 1996, *The Java Language Specification* (1st ed.; Boston, MA: Addison-Wesley)
- Gosling, J., Joy, B., Steele, G. L., Bracha, G., & Buckley, A. 2014, *The Java Language Specification, Java SE 8 Edition* 1st ed. (Addison-Wesley)
- Kernighan, B. W. 1984, *The UNIX Programming Environment* (Englewood Cliffs, NJ: Prentice-Hall)
- Kernighan, B. W., & Ritchie, D. M. 1988, *The C Programming Language* (2nd ed; Englewood Cliffs, NJ: Prentice-Hall)
- Kluyver, T., Ragan-Kelley, B., Perez, F., et al. 2016, *Jupyter Notebooks: A Publishing Format for Reproducible Computational Workflows* (Amsterdam: IOS Press)
- Langtangen, H. P. 2009, *Python Scripting for Computational Science* (3rd ed.; Berlin: Springer)
- Perez, F., & Granger, B. E. 2007, *CSE*, **9**, 21
- Press, W. H., Teukolsky, S., Saul, A., Vetterling, W., William, T., & Flannery, B. P. 1992, *Numerical Recipes in C. The Art of Scientific Computing* (Cambridge: Cambridge Univ. Press)

² If you do not know what a segmentation fault is, make a point of never finding out (after Press et al. 1992 in relation to the bubble-sort algorithm).

- Rojas, S. J. G., Christensen, E. A., & Blanco-Silva, F. J. 2015, Learning SciPy for Numerical and Scientific Computing Second Edition (2nd ed.; Birmingham: Packt Publishing)
- Sterling, T. L. 2002, Beowulf Cluster Computing with Linux (Cambridge, MA: MIT Press)
- Stroustrup, B. 2013, The C++ Programming Language (4th ed.; Reading, MA: Addison-Wesley)
- The Astropy CollaborationRobitaille, T. P., Tollerud, E. J., et al. 2013, [A&A](#), [558](#), [A33](#)
- van der Walt, S., Colbert, S. C., & Varoquaux, G. 2011, [CSE](#), [13](#), [22](#)
- van Rossum, G. 1995a, Extending and Embedding the Python Interpreter CS-R9527, *CWI*
- van Rossum, G. 1995b, Python Library Reference CS-R9524, *CWI*
- van Rossum, G. 1995c, Python Reference Manual CS-R9525, *CWI*
- van Rossum, G. 1995d, Python Tutorial CS-R9526, *CWI*